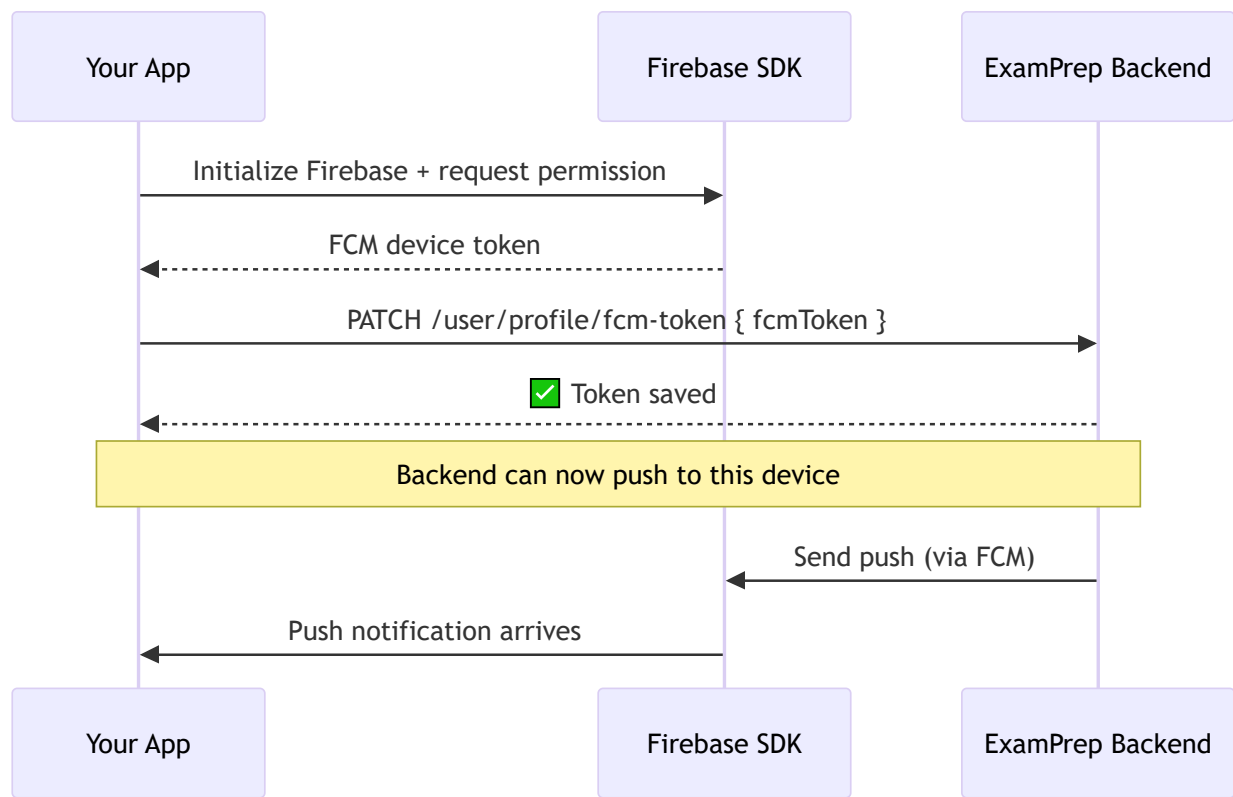


-  [Notifications — Frontend Integration Guide](#)
 - [Quick Start](#)
 - [API Endpoints](#)
 - [1. Register / Update FCM Token](#)
 - [2. Get Notifications](#)
 - [3. Mark Notification as Read](#)
 - [4. Mark All Notifications as Read](#)
 - [4. Admin: Broadcast Notification](#)
 - [5. Admin: Targeted Notification](#)
 - [Push Notification Payload](#)
 - [Notification Types & Recommended UI](#)
 - [Client-Side Integration](#)
 - [Web App \(Firebase JS SDK\)](#)
 - [Mobile App \(React Native\)](#)
 - [Flutter](#)
 - [In-App Notification UI Checklist](#)
 - [Backend Notes \(For Context\)](#)

Notifications — Frontend Integration Guide

API reference and integration guide for **push notifications** (Firebase Cloud Messaging) and **in-app notifications** in the ExamPrep platform.

Quick Start



3 steps to enable push notifications:

1. Initialize Firebase SDK in your app and obtain an FCM token
2. Register the token with the backend via the API
3. Handle incoming push messages in your app

API Endpoints

1. Register / Update FCM Token

Call this **on every app launch** and whenever the FCM token refreshes.

Method	PATCH
URL	/user/profile/fcm-token
Auth	Bearer <jwt>

Request:

```
{
  "fcmToken": "dK3x...your-device-fcm-token"
```

```
}
```

Response 200 OK:

```
{
  "message": "FCM token updated successfully"
}
```

[!IMPORTANT] You **must** call this endpoint every time the FCM token changes (e.g. app reinstall, token refresh callback). Stale tokens cause silent push failures.

2. Get Notifications

Fetches a paginated list of in-app notifications for the authenticated student.

Method	GET
URL	/user/notifications
Auth	Bearer <jwt>
Query Params	page (default 1), limit (default 10), type (optional)

Response 200 OK:

```
{
  "data": [
    {
      "id": "clxyz...",
      "studentId": "uuid-student",
      "title": "Achievement Unlocked! 🏆",
      "message": "You've unlocked the \"First Timer\" achievement!",
      "type": "ACHIEVEMENT",
      "data": { "achievementName": "First Timer" },
      "readAt": null,
      "createdAt": "2026-02-18T06:00:00.000Z"
    }
  ],
  "meta": {
    "total": 45,
    "page": 1,
    "limit": 10,
    "totalPages": 5
  }
}
```

```
}  
}
```

3. Mark Notification as Read

Method	PATCH
--------	-------

URL	/user/notifications/:id/read
-----	------------------------------

Auth	Bearer <jwt>
------	--------------

Response 200 OK:

```
{  
  "success": true,  
  "message": "Notification marked as read"  
}
```

4. Mark All Notifications as Read

Method	PATCH
--------	-------

URL	/user/notifications/read-all
-----	------------------------------

Auth	Bearer <jwt>
------	--------------

Response 200 OK:

```
{  
  "success": true,  
  "message": "All notifications marked as read"  
}
```

4. Admin: Broadcast Notification

Sends a notification to **every** student (in-app + push).

Method	POST
---------------	------

URL	/admin/promotion/broadcast
------------	----------------------------

Auth	Bearer <admin-jwt>
-------------	--------------------

Request:

```
{
  "title": "New Feature Available!",
  "message": "Check out our latest exam prep tools",
  "type": "PROMOTION",
  "data": { "promoCode": "SAVE20", "deepLink": "/promotions/save20" }
}
```

Response 201:

```
{
  "successCount": 150,
  "total": 155
}
```

5. Admin: Targeted Notification

Sends to students matching filters.

Method	POST
---------------	------

URL	/admin/promotion/targeted
------------	---------------------------

Auth	Bearer <admin-jwt>
-------------	--------------------

Request:

```
{
  "filters": {
    "examType": "WAEC",
    "academicLevel": "SSS3",
    "universityId": "uuid-here"
  }
}
```

```
},
"title": "WAEK Prep Special",
"message": "30% off all WAEK practice exams!",
"type": "PROMOTION",
"data": {}
}
```

Response: Same shape as broadcast.

Push Notification Payload

When a push notification arrives on the device, the FCM message has this structure:

```
{
  "notification": {
    "title": "Achievement Unlocked! 🏆",
    "body": "You've unlocked the \"First Timer\" achievement!"
  },
  "data": {
    "achievementName": "First Timer"
  }
}
```

Field	Type	Description
<code>notification.title</code>	<code>string</code>	Display title
<code>notification.body</code>	<code>string</code>	Display body text
<code>data</code>	<code>object</code>	Custom key-value pairs for deep linking / UI logic

[!TIP] Use the `data` payload for **deep linking**. For example, when `type = TASK_ASSIGNED`, `data` contains `{ taskId }` — navigate the user to the task detail screen.

Notification Types & Recommended UI

Use the `type` field to style notifications and handle navigation:

Type	Icon	Example Title	data Payload	Suggested Navigation
ACHIEVEMENT		Achievement Unlocked!	{ achievementName }	→ Achievements screen
STREAK_REMINDER		Maintain your streak!	{ currentStreak }	→ Practice / Home
STREAK_LOST		Streak Lost	(none)	→ Practice / Home
STREAK_FROZEN		Streak Frozen	{ remainingFreezes }	→ Profile / Streaks
TASK_ASSIGNED		New Task Assigned!	{ taskId }	→ Task detail screen
PROMOTION		(varies)	(varies, e.g. promoCode)	→ Promo screen / deep link

Client-Side Integration

Web App (Firebase JS SDK)

```
// 1. Initialize Firebase
import { initializeApp } from 'firebase/app';
import { getMessaging, getToken, onMessage } from 'firebase/messaging';

const app = initializeApp({ /* your firebase config */ });
const messaging = getMessaging(app);

// 2. Request permission & get token
async function registerForPush(jwtToken) {
  const permission = await Notification.requestPermission();
  if (permission !== 'granted') return;

  const fcmToken = await getToken(messaging, {
    vapidKey: 'YOUR_VAPID_KEY'
  });
}
```

```
// 3. Send token to backend
await fetch('/user/profile/fcm-token', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${jwtToken}`
  },
  body: JSON.stringify({ fcmToken })
});
}

// 4. Handle foreground messages
onMessage(messaging, (payload) => {
  // Show in-app toast / notification badge
  console.log('Push received:', payload.notification);
  // Update notification bell badge count
});
```

[!NOTE] You also need a **service worker** ([firebase-messaging-sw.js](#)) for background push handling. See the [Firebase docs](#).

Mobile App (React Native)

```
// Using @react-native-firebase/messaging
import messaging from '@react-native-firebase/messaging';

// 1. Request permission (iOS)
await messaging().requestPermission();

// 2. Get token
const fcmToken = await messaging().getToken();

// 3. Register with backend
await api.patch('/user/profile/fcm-token', { fcmToken });

// 4. Listen for token refresh
messaging().onTokenRefresh(async (newToken) => {
  await api.patch('/user/profile/fcm-token', { fcmToken: newToken });
});

// 5. Handle foreground messages
messaging().onMessage(async (remoteMessage) => {
  // Show local notification or update UI
  const { title, body } = remoteMessage.notification;
  const data = remoteMessage.data; // { type, taskId, etc. }
});

// 6. Handle background/quit state (in index.js)
messaging().setBackgroundMessageHandler(async (remoteMessage) => {
```

```
// Handle silently or navigate on tap
});
```

Flutter

```
// Using firebase_messaging package
import 'package:firebase_messaging/firebase_messaging.dart';

// 1. Request permission
await FirebaseMessaging.instance.requestPermission();

// 2. Get token & register
final token = await FirebaseMessaging.instance.getToken();
await api.patch('/user/profile/fcm-token', {'fcmToken': token});

// 3. Token refresh
FirebaseMessaging.instance.onTokenRefresh.listen((newToken) {
  api.patch('/user/profile/fcm-token', {'fcmToken': newToken});
});

// 4. Foreground messages
FirebaseMessaging.onMessage.listen((RemoteMessage message) {
  // Show local notification / update badge
});

// 5. Background handler (top-level function)
FirebaseMessaging.onBackgroundMessage(_backgroundHandler);
```

In-App Notification UI Checklist

Use this checklist when building the notification center:

- ☐ **Bell icon** with unread count badge (count notifications where `readAt` is `null`)
 - ☐ **Notification list** — sorted newest first, with unread highlight
 - ☐ **Read/unread toggle** — call mark-as-read endpoint on tap
 - ☐ **Pull-to-refresh** — re-fetch from API
 - ☐ **Type-based icons** — map `type` field to emoji/icon (see table above)
 - ☐ **Deep linking** — use `data` payload to navigate on tap
 - ☐ **Empty state** — "No notifications yet" illustration
 - ☐ **Pagination** — infinite scroll or load-more button
-

Backend Notes (For Context)

What	Detail
Push provider	Firebase Admin SDK via <code>firebase-admin</code> npm package
In-app storage	PostgreSQL <code>in_app_notifications</code> table
Delivery guarantee	In-app is guaranteed (DB write); push is best-effort (failures are logged, not retried)
Token storage	<code>fcmToken</code> column on <code>students</code> table
Batch notifications	Admin broadcasts use <code>Promise.allSettled</code> — partial failures are acceptable
Config env var	<code>FIREBASE_SERVICE_ACCOUNT_PATH</code> — path to the service account JSON